

EMBEDDED SYSTEMS
REAL TIME BATTERY MONITORING

Adaptive Multi-Cell Battery Intelligence Engine

BACHELOR OF TECHNOLOGY
in
ELECTRONICS AND COMMUNICATION
ENGINEERING

Submitted by
Sudanapalli Srinivas

Srinivas79798@gmail.com

CONTENTS:

1. Introduction to Embedded systems
2. Literature review
3. Battery management system
4. Pin configurations
5. ADC
6. Relay and Buzzer
7. Testing and Debugging
8. Code
9. Conclusion

ABSTRACT

The Adaptive Multi-Cell Battery Intelligence Engine is an embedded system designed to monitor and analyze the performance of a simulated 4-cell lithium-ion battery pack in real time. The system continuously acquires voltage data from individual battery cells through analog inputs and processes the information to evaluate overall battery pack health. Key parameters such as individual cell voltages, average pack voltage, cell imbalance percentage, strongest cell, and weakest cell are calculated to provide comprehensive battery diagnostics.

The intelligence engine classifies battery conditions into predefined health states including Healthy, Minor Imbalance, Critical Imbalance, and Pack Failure based on configurable threshold values. A modular and scalable software architecture is implemented to ensure easy maintenance, future expansion, and efficient real-time operation. The system also supports remote monitoring through IoT connectivity, enabling live visualization of battery parameters and health status on a cloud-based dashboard.

This project demonstrates the application of embedded systems, real-time data processing, battery management concepts, and IoT technologies to create a reliable and intelligent battery monitoring solution suitable for energy storage systems, electric vehicles, and other battery-powered applications.

1. INTRODUCTION TO EMBEDDED SYSTEMS

Embedded systems are specialized computer systems designed to perform specific tasks within electronic devices. They consist of hardware components such as microcontrollers, sensors, memory, and software that work together to control and monitor operations. Unlike general-purpose computers, embedded systems are optimized for efficiency, reliability, and real-time performance.

These systems are widely used in applications such as automotive electronics, industrial automation, healthcare devices, consumer electronics, and IoT solutions. In battery monitoring systems, embedded systems collect and process real-time data such as voltage, current, and temperature, enabling efficient battery management, improved safety, and enhanced system performance.

2. LITERATURE REVIEW

Battery monitoring and management systems have become increasingly important with the widespread use of lithium-ion batteries in electric vehicles, renewable energy storage systems, and portable electronic devices. Traditional battery monitoring methods mainly focused on measuring overall pack voltage and current, which provided limited information about the condition of individual cells. Research studies have shown that monitoring each cell separately helps detect voltage imbalances, improves battery safety, and extends battery lifespan.

Recent developments in embedded systems and IoT technologies have enabled real-time battery monitoring and intelligent health analysis. Modern Battery Management Systems (BMS) use microcontrollers, sensors, and wireless communication modules to collect and transmit battery data continuously. These systems can identify weak or faulty cells, calculate imbalance percentages, and classify battery health conditions, allowing preventive maintenance and improved reliability. The concepts and technologies presented in previous studies form the foundation for developing the Adaptive Multi-Cell Battery Intelligence Engine used in this project.

3. BATTERY MANAGEMENT SYSTEM (BMS)

A Battery Management System (BMS) is an electronic system that monitors, controls, and protects rechargeable battery packs during operation. Its primary function is to ensure the safe and efficient performance of batteries by continuously tracking parameters such as cell voltage, current, temperature, and overall battery health. A BMS helps prevent conditions such as overcharging, over-discharging, overheating, and cell imbalance, which can reduce battery life and compromise safety.

In multi-cell battery packs, the BMS plays a crucial role in monitoring individual cell voltages and identifying weak or strong cells. It analyzes battery conditions, detects abnormalities, and provides health status information to users or connected systems. In this project, the Battery Management System monitors a 4-cell lithium battery pack in real time, calculates average voltage and cell imbalance, and classifies battery health into states such as Healthy, Minor Imbalance, Critical Imbalance, and Pack Failure. This improves battery reliability, safety, and overall performance.

4. PIN CONFIGURATIONS

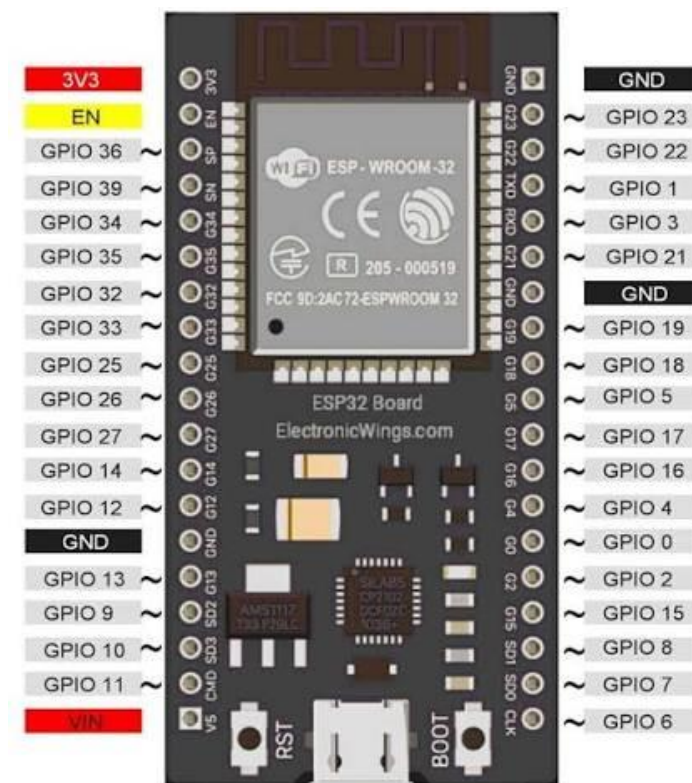


Fig 1 ESP32

TYPES	COMPONENTS	FUNCTION
Analog Input	Cell Voltage Sensors	Measure individual cell voltages
Analog Input	NTC Thermistor	Measure battery temperature
Analog Input	Hall Effect Sensor	Measure charging/discharging current
Digital Output	Fault LED	Indicates battery faults
Digital Output	Relay/Contactor	Connects or disconnects battery pack
Digital Output	Buzzer	Provides audible fault alerts

Fig 2 list of I/p & O/p

5. ADC

Working Steps

- Read analog voltage from battery cells.
- Convert analog signal into a digital value (0–4095 for 12-bit ADC).
- Calculate actual cell voltage.
- Analyze battery pack condition.
- Display and transmit results to the IoT dashboard.

Example formula used in ESP32:
Voltage = (ADC_Value / 4095.0) × 3.3

Where:

- ADC_Value = Digital value read from ADC pin.
- 4095 = Maximum value of 12-bit ADC.
- 3.3V = Reference voltage of ESP32

6. RELAY AND BUZZER

Functions

RELAY

- Connects or disconnects the battery pack.
- Protects the system during fault conditions.
- Controlled by ESP32 digital output.

Buzzer

- Generates audible warnings.
- Alerts users about battery faults.
- Activated during critical battery conditions.

7. TESTING AND DEBUGGING

- Verified ADC readings for all four battery cells.
- Tested voltage calculation and averaging logic.
- Checked strongest and weakest cell identification.
- Validated battery health classification.
- Tested IoT dashboard data updates.
- Verified relay and buzzer operation during fault conditions.
- Debugged code using Serial Monitor output.
- Confirmed stable real-time system performance.

8. CODE

```
#define BLYNK_TEMPLATE_ID "TMPL3qvgvLIHh"

#define BLYNK_TEMPLATE_NAME "Adaptive MutliCell Battery Intelligence Engine"

#define BLYNK_AUTH_TOKEN "6BOhEgHbhLhzEME7WD-cbJlN2sIi7rRf"


#include <WiFi.h>

#include<BlynkSimpleEsp32.h>

char ssid[]= "Wokwi-GUEST";

char pass[]="";


#include <Wire.h>

#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define CELL1 34

#define CELL2 35
```

```
#define CELL3 32

#define CELL4 33

#define RED_LED 2

#define GREEN_LED 4

#define YELLOW_LED 5

#define BUZZER 18

#define RELAY 19

float v1,v2,v3,v4;

float averagevoltage;

float maxvoltage, minvoltage;

float imbalance;

String status;

String strongestcell;

String weakestcell;

float readvoltage(int pin){

    int raw=analogRead(pin);

    return (raw/4095.0)*3.3;

}

void setup() {

    // put your setup code here, to run once:

    Serial.begin(115200);

    pinMode(RED_LED, OUTPUT);
```

```

pinMode(GREEN_LED, OUTPUT);

pinMode(YELLOW_LED, OUTPUT);

pinMode(BUZZER, OUTPUT);

pinMode(RELAY, OUTPUT);

lcd.init();

lcd.backlight();

digitalWrite(RELAY, LOW);

Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);


Serial.println("---Adaptive Mutli-Cell Battery Intelligence Engine
started---");

}


void loop() {

v1=readvoltage(CELL1);

v2=readvoltage(CELL2);

v3=readvoltage(CELL3);

v4=readvoltage(CELL4);


averagevoltage=(v1+v2+v3+v4)/4.0;

maxvoltage= max(max(v1,v2), max(v3, v4));

minvoltage= min(min(v1,v2), min(v3, v4));

if (maxvoltage==v1)

```

```
    strongestcell="CELL1";  
else if(maxvoltage==v2)  
    strongestcell="CELL2";  
else if(maxvoltage==v3)  
    strongestcell="CELL3";  
else  
    strongestcell="CELL4";  
if(minvoltage==v1)  
    weakestcell="CELL1";  
else if(minvoltage==v2)  
    weakestcell="CELL2";  
else if(minvoltage==v3)  
    weakestcell="CELL3";  
else  
    weakestcell="CELL4";  
imbalance=(maxvoltage-minvoltage)/maxvoltage*100;
```

```
digitalWrite(RED_LED, LOW);  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(YELLOW_LED, LOW);  
digitalWrite(BUZZER, LOW);
```

```
if(minvoltage<1.0){  
    status= "PACK FAILURE";  
    digitalWrite(RED_LED, HIGH);  
    digitalWrite(BUZZER, HIGH);  
    digitalWrite(RELAY, HIGH);  
}  
else if(imbalance>25){  
    status="CRITICAL IMBALANCE";  
    digitalWrite(RED_LED, HIGH);  
    digitalWrite(BUZZER, HIGH);  
}  
else if(imbalance>10){  
    status="MINOR IMBALANCE";  
    digitalWrite(YELLOW_LED, HIGH);  
}  
else{  
    status="HEALTHY";  
    digitalWrite(GREEN_LED, HIGH);  
}  
Serial.println("...BATTERY CONDITION...");  
Serial.print("CELL1: "); Serial.print(v1,2); Serial.println("V");  
Serial.print("CELL2: "); Serial.println(v2);
```

```
Serial.print("CELL3: "); Serial.println(v3);  
Serial.print("CELL4: "); Serial.println(v4);  
Serial.print("Pack averages: "); Serial.println(averagevoltage);  
Serial.print("Maximum voltage: "); Serial.println(maxvoltage);  
Serial.print("Minimum voltage: "); Serial.println(minvoltage);  
Serial.print("Strongest cell: "); Serial.println(strongestcell);  
Serial.print("Weakest cell: "); Serial.println(weakestcell);  
Serial.print("Imbalance Percentage: "); Serial.println(imbalance);  
Serial.print("Battery status: "); Serial.println(status);  
Serial.println();  
lcd.clear();  
lcd.setCursor(0,0);  
lcd.print("AVG:");  
lcd.print(averagevoltage, 1);  
lcd.setCursor(0,1);  
lcd.print(status);  
  
Blynk.run();  
  
Blynk.virtualWrite(V0, v1);  
Blynk.virtualWrite(V1, v2);  
Blynk.virtualWrite(V2, v3);
```

```
Blynk.virtualWrite(V3, v4);
```

```
Blynk.virtualWrite(V4, averagevoltage);
```

```
Blynk.virtualWrite(V5, maxvoltage);
```

```
Blynk.virtualWrite(V6, minvoltage);
```

```
Blynk.virtualWrite(V7, imbalance);
```

```
Blynk.virtualWrite(V8, strongestcell);
```

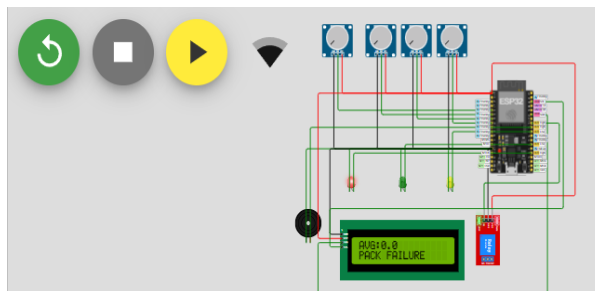
```
Blynk.virtualWrite(V9, weakestcell);
```

```
Blynk.virtualWrite(V10, status);
```

```
  delay(1000);
```

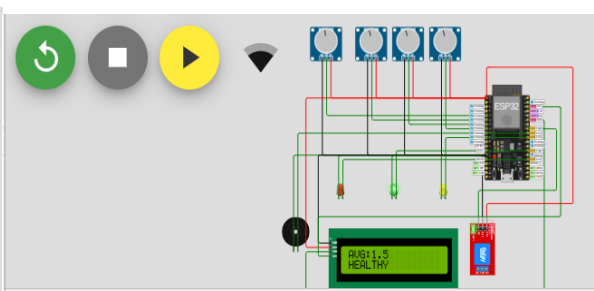
```
}
```

OUTPUTS



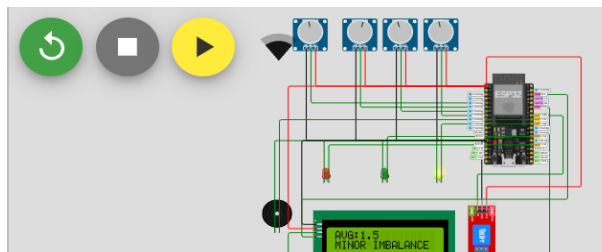
```
...BATTERY CONDITION...
CELL1: 0.00V
CELL2: 0.00
CELL3: 0.00
CELL4: 0.00
Pack averages: 0.00
Maximum voltage: 0.00
Minimum voltage: 0.00
Strongest cell: CELL1
Weakest cell: CELL1
Imbalance Percentage: nan
Battery status: PACK FAILURE
```

Fig 3 pack failure



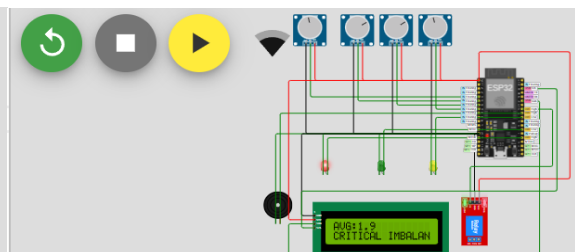
```
...BATTERY CONDITION...
CELL1: 1.52V
CELL2: 1.48
CELL3: 1.45
CELL4: 1.48
Pack averages: 1.48
Maximum voltage: 1.52
Minimum voltage: 1.45
Strongest cell: CELL1
Weakest cell: CELL3
Imbalance Percentage: 4.04
Battery status: HEALTHY
```

Fig 4 healthy



```
...BATTERY CONDITION...
CELL1: 1.52V
CELL2: 1.72
CELL3: 1.45
CELL4: 1.48
Pack averages: 1.54
Maximum voltage: 1.72
Minimum voltage: 1.45
Strongest cell: CELL2
Weakest cell: CELL3
Imbalance Percentage: 15.58
Battery status: MINOR IMBALANCE
```

Fig 5 minor imbalance



```
...BATTERY CONDITION...
CELL1: 1.52V
CELL2: 2.37
CELL3: 2.34
CELL4: 1.48
Pack averages: 1.92
Maximum voltage: 2.37
Minimum voltage: 1.48
Strongest cell: CELL2
Weakest cell: CELL4
Imbalance Percentage: 37.70
Battery status: CRITICAL IMBALANCE
```

Fig 6 critical imbalance

9. CONCLUSION

The Adaptive Multi-Cell Battery Intelligence Engine successfully monitors a simulated 4-cell lithium-ion battery pack in real time using the ESP32 microcontroller. The system measures individual cell voltages, calculates pack statistics, identifies the strongest and weakest cells, and determines battery health conditions. Through IoT integration, real-time battery data is displayed on a cloud dashboard, enabling remote monitoring and analysis. The project demonstrates the effective application of embedded systems, battery management principles, and IoT technologies for developing intelligent and reliable battery monitoring solutions.

THANK YOU